

NumPy 데이터 추출과 통계 분석 가이드

인덱싱/슬라이싱, 벡터 연산, 조건 필터링 및 axis 기반 기초 통계 분석 완전 정복

1. 데이터 추출의 핵심: 인덱싱과 슬라이싱

배열을 생성하고 구조를 이해한 뒤에는 필요한 데이터만 콧 집어 꺼내는 기술이 필요합니다. NumPy는 1차원 데이터뿐만 아니라 2차원 표 구조에서도 매우 직관적인 추출 방식을 제공합니다.

① 1차원 인덱싱 & 슬라이싱

인덱스는 **0부터 시작**하며, **-1** 을 사용하면 배열의 맨 마지막 요소를 가리킵니다. 슬라이싱 규칙(**[시작:끝:간격]**)에서는 '**시작은 포함, 끝은 제외**'라는 파이썬 표준 약속이 동일하게 적용됩니다.

```
# 1차원 배열 인덱싱 및 슬라이싱
temp = np.array([70, 72, 71, 95, 73, 68])

print(temp[0])      # 70 (첫 번째)
print(temp[-1])     # 68 (맨 마지막)
print(temp[1:4])    # [72 71 95] (1, 2, 3번 인덱스, 4번 제외)
print(temp[:2])     # [70 71] (두 칸씩 건너뛰기)
```

② 2차원 인덱싱 & 슬라이싱 (행, 열 지정)

2차원 표 형태에서는 쉼표(,)를 기준으로 **[행 인덱스, 열 인덱스]** 순서로 접근합니다. 모든 행 또는 모든 열을 선택할 때는 콜론(:) 기호를 사용합니다.

추출 목적	문법 및 표현	코드 예시 (`data`가 2차원 표일 때)	설명 및 활용
특정 칸 하나	<code>data[행, 열]</code>	<code>data[0, 1]</code>	0행 1열 위치의 단일 데이터 추출
행 전체 (특정 시점)	<code>data[행]</code>	<code>data[0]</code>	0번째 시점의 모든 센서 데이터 추출
열 전체 (특정 센서)	<code>data[:, 열]</code>	<code>data[:, 0]</code>	모든 시점에 대한 0번 센서의 데이터 전체 추출

2. 배열 연산과 조건 필터링

① 벡터 연산과 브로드캐스팅 (Broadcasting)

NumPy에서는 **for** 반복문 없이 배열 간 요소별 계산(Element-wise)이 수행됩니다. 크기가 다른 배열이나 숫자 하나(스칼라)를 연산할 때는 작은 크기의 배열이 큰 배열 모양으로 확장되어 계산되는 ****브로드캐스팅****이 일어납니다.

```
# 스칼라 연산 (단위 변환)
celsius = np.array([20.0, 25.0, 30.0])
fahrenheit = celsius * 1.8 + 32 # [68. 77. 86.]

# 브로드캐스팅 (2차원 표의 각 행에서 1차원 기준값 뺄셈)
table = np.array([[72, 2.3], [95, 6.8]])
base = np.array([70, 2.0])
diff = table - base # [[2, 0.3], [25, 4.8]]
```

② 조건 필터링 (Boolean Indexing & np.where)

조건식을 작성하면 `True/False` 로 구성된 불리언 배열이 만들어집니다. 이를 활용하면 이상 센서값만 선별하거나 위험 고리표를 붙일 수 있습니다.

- **Boolean Indexing:** `arr[조건식]` 형태. 조건이 `True` 인 위치의 값만 골라내며 결과 크기가 줄어듭니다.
- **`np.where(조건식, 참일때값, 거짓일때값)`:** 조건에 따라 값을 치환하며 원본 크기와 형태가 유지됩니다.
- **다중 조건:** 각각의 조건을 괄호로 묶고 `&` (AND) 또는 `|` (OR) 기호를 사용합니다. (예: `(arr > 70) & (arr < 90)`)

3. 기초 통계 분석과 축(axis) 개념

수만 개의 데이터를 몇 개의 대표 숫자로 요약하여 설비의 정상 상태를 정의하고 이상 징후를 탐지합니다.

통계 함수	주요 의미	특징 및 실무 활용
<code>mean()</code> / <code>np.median()</code>	평균 / 중앙값	중앙값은 극단적 이상치에 흔들리지 않음 (두 값의 차이로 이상치 유무 점검)
<code>max()</code> / <code>min()</code>	최대 / 최소값	작동 범위 및 위험했던 최고 순간 확인
<code>std()</code> / <code>var()</code>	표준편차 / 분산	데이터가 흩어진 정도. 표준편차가 갑자기 커지면 설비 불안정 신호

🎯 2차원 통계의 핵심: axis (축) 방향 구분하기

- **`axis=0` (열별 / 센서별 통계):** 위에서 아래로 행을 따라 계산 → 각 센서(열)의 평균/표준편차 반환 (가장 자주 사용!)
- **`axis=1` (행별 / 시점별 통계):** 왼쪽에서 오른쪽으로 열을 따라 계산 → 각 시점(행)에서의 설비 통합 수치 반환

4. 데이터 분석 5단계 전체 워크플로우

NumPy 데이터 분석은 항상 다음 5단계의 일관된 흐름으로 진행됩니다.

1. 불러오기 (`np.loadtxt`): 텍스트/CSV 파일에서 배열로 데이터를 불러옵니다.
2. 구조 확인 (`shape`, `dtype`): 차원과 데이터 타입을 점검합니다.
3. 추출 (`data[:, k]`): 슬라이싱을 통해 원하는 센서 열만 분리합니다.
4. 필터링 (`arr[arr > 기준]`): 조건을 이용해 위험/이상 시점을 선별합니다.
5. 통계 요약 (`mean()`, `std()`): 평균과 표준편차로 이상 시점의 특성을 요약합니다.